

Stanford CS149: Parallel Computing

Written Assignment 1

Multi-Core Architecture

Problem 1:

- A. Consider a multi-core processor that runs at 2 GHz and has 4 cores. Each core can perform up to one 8-wide SIMD vector instruction per clock and supports hardware multi-threading with 4 hardware execution context per core. What is the maximum throughput of the processor in units of scalar floating point operations per second? Please show your calculations.

freq. = 2GHz, 4 cores, 8-wide SIMD vector instruction/clock

max. throughput in units of scalar = $2G * 4 * 8 = 64 G \text{ ops/sec}$

64 GFLOPS

Note: multi-threading job is to make the throughput reach the max. throughput as much as possible

- B. Consider the processor from part A running a program that perfectly parallelizes onto many cores, makes perfect usage of SIMD vector processing, and has an arithmetic intensity of 4. (4 scalar floating ops per byte of data transferred from memory.) If we assume the processor has a memory system providing 64 GB/sec of bandwidth, is the program compute-bound or bandwidth bound on this processor? You may assume for math simplicity that 1 GB/sec is a one billion bytes per second (10^9). Please show your work.

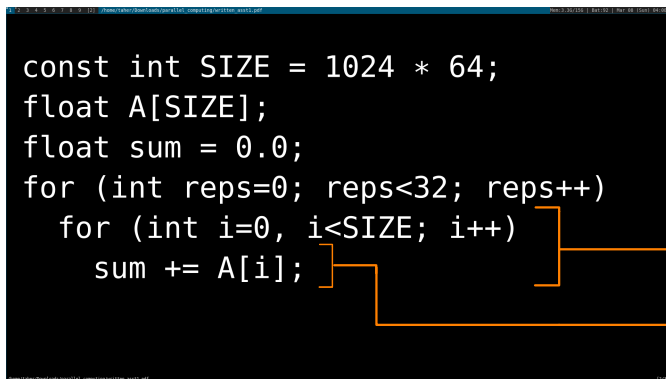
freq. = 2GHz, 4 cores, 8-wide SIMD vector instruction/clock

we've 4 arithmetic/byte, which means half of the data we get from the memory are arithmetic ops. from A, we can do 64 G ops/sec and we can get 64GB/sec with 48 floating point operations

Since, we've arithmetic intensity of 4, and we can do 64 GFLOPS, we've 64

- C. Consider a cache that contains 32 KB of data, has a cache line size of 4 bytes, is fully associative (meaning any cache line can go anywhere in the cache), and uses an LRU (least recently used—the line evicted is the line that was last accessed the longest time ago) replacement policy. Please describe why the following code will take a cache miss on every data access to the array A.

```
const int SIZE = 1024 * 64;  
float A[SIZE];  
float sum = 0.0;  
for (int reps=0; reps<32; reps++)  
    for (int i=0, i<SIZE; i++)  
        sum += A[i];
```



cache size = 32KB, cache line size = 4 bytes

so, we've $32/4 = 8K$ cache lines

repeat this 32 times. I think we do this 'cause float is 32bit? I see no reason for doing this honestly

repeat this for all the array elements

Every data access to the array A, we get 4 bytes(one float) or one element of the array

So, because we're doing this with each element of the array we'll get a cold miss every time

we do this $1024*64$ at the first reps, so we replace 65,536 cache line, but actually we just have 8K so, actually we'll have misses within the first repetition. so, the next reps we'll cache misses every time too.

test test2

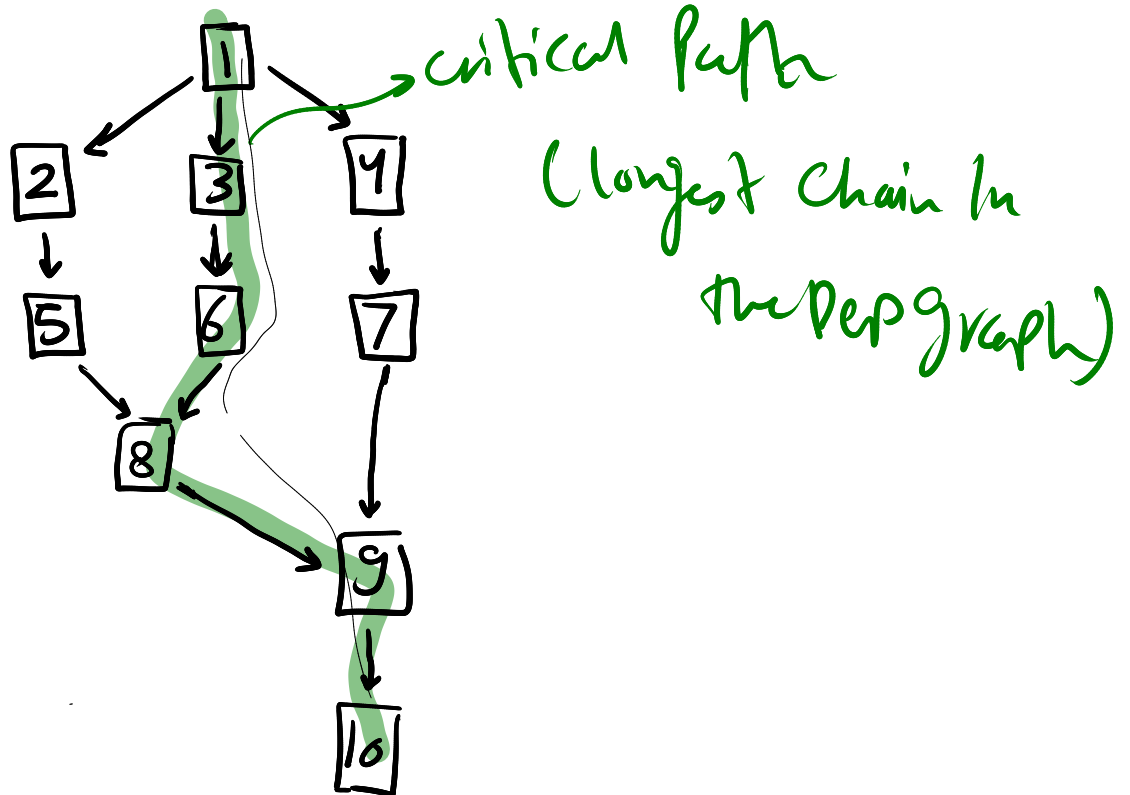
Dependency Graphs, ILP, and Superscalar/Multi-Threaded Execution

Problem 2:

Consider the following sequence of scalar instructions running within a single thread.

```
1. LD  R0 <- mem[R4]    // load memory address given by R4 into R0
2. ADD R1 <- R0, R0      // R1 = R0 + R0
3. ADD R2 <- R0, R0      // R2 = R0 + R0
4. ADD R3 <- R0, R0      // R3 = R0 + R0
5. MUL R1 <- R1, R1
6. MUL R2 <- R2, R2
7. MUL R3 <- R3, R3
8. ADD R1 <- R1, R2
9. ADD R1 <- R1, R3
10. ST  mem[R5] <- R1    // store R1 into memory at address given by R5
```

A. Please draw the instruction dependency graph for this instruction sequence.

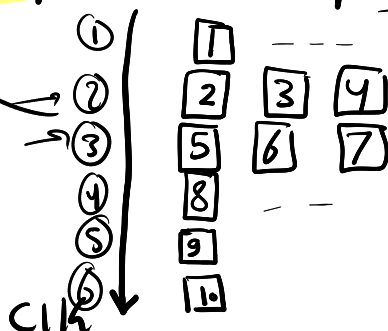


max IPC = 3

IPC > 1

B. What is the maximum amount of instruction level parallelism (ILP) present in the program. Keep in mind that ILP is entirely a property of the program itself, not the machine it is run on.

ILP makes use of Independent Instructions



ILP depends on the Critical Path.

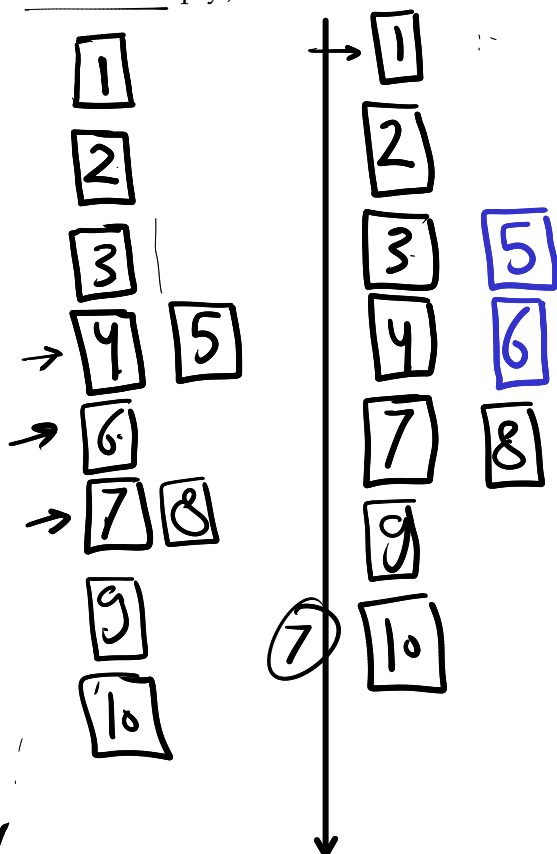
$$IPL = \frac{\text{No Instr}}{\text{Critical Path}}$$

1. LD R0 <- mem[R4]
2. ADD R1 <- R0, R0
3. ADD R2 <- R0, R0
4. ADD R3 <- R0, R0
5. MUL R1 <- R1, R1
6. MUL R2 <- R2, R2
7. MUL R3 <- R3, R3
8. ADD R1 <- R1, R2
9. ADD R1 <- R1, R3
10. ST mem[R5] <- R1

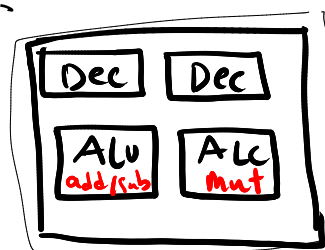
$$= \frac{10}{6} = 1.6 \times$$

C. Consider running this instruction stream on a single core, single-threaded processor that has superscalar execution capability to perform up to two instructions per clock, PROVIDED THAT EXACTLY ONE INSTRUCTION IS A MUL. In other words, the processor has two execution units (ALUs): one can execute add/load/store instructions and the other can execute only multiplications. Please assume all instructions (including loads/stores) individually complete in one cycle. You cannot modify the instructions in the program. What is the minimum number of cycles needed to execute this program? (Hint: think carefully about what instructions are allowed to run concurrently. The processing can run two instructions at the same time if they are independent and exactly one is a multiply.)

clk's



1. LD R0 <- mem[R4]
2. ADD R1 <- R0, R0
3. ADD R2 <- R0, R0
4. ADD R3 <- R0, R0
5. MUL R1 <- R1, R1
6. MUL R2 <- R2, R2
7. MUL R3 <- R3, R3
8. ADD R1 <- R1, R2
9. ADD R1 <- R1, R3
10. ST mem[R5] <- R1



8 clk cycles
(In-order)

7 clk cycles
(Out-of-order)

D. Now assume that the code above is changed so that it is: (1) running in a loop, where instructions 1-10 make up the body of the loop, and in each iteration of the loop the addresses stored in R4 and R5 are different. (2) Loop iterations are perfectly parallelized across 4 threads. We are running this code on a single core, multi-threaded processor that can process one instruction per clock (arithmetic instructions, LDs, STs). However, from the moment a processor begins to execute a load instruction, there is a latency of 25 cycles before the data can be used by a dependent instruction. To be clear: if a core issues a LD in clock c , the LD occupies the core in clock c , but then the core cannot run an instruction that uses the value of the load until clock $c + 25$.

How many threads must be interleaved for the core to run at 100% efficiency? Please justify your answer.

means Processor never Stalls (utilization)

Old Code

```

1. LD R0 <- mem[R4]
2. ADD R1 <- R0, R0
3. ADD R2 <- R0, R0
4. ADD R3 <- R0, R0
5. MUL R1 <- R1, R1
6. MUL R2 <- R2, R2
7. MUL R3 <- R3, R3
8. ADD R1 <- R1, R2
9. ADD R1 <- R1, R3
10. ST mem[R5] <- R1

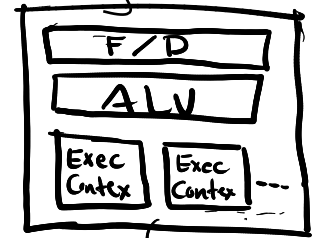
```

New Code

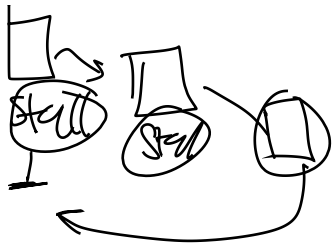
```

for (j=0; j<N; j++) {
1. LD R0 <- mem[R4+j]
2. ADD R1 <- R0, R0
3. ADD R2 <- R0, R0
4. ADD R3 <- R0, R0
5. MUL R1 <- R1, R1
6. MUL R2 <- R2, R2
7. MUL R3 <- R3, R3
8. ADD R1 <- R1, R2
9. ADD R1 <- R1, R3
10. ST mem[R5+j] <- R1
}

```



→ Question is "How many?"

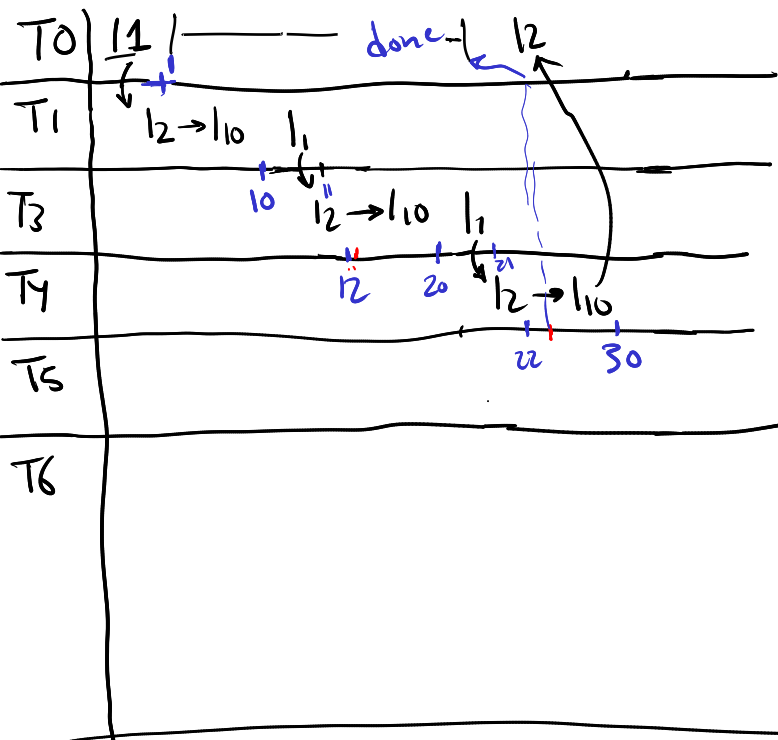


In the case of the first instruction, we can resume the thread at clock cycle 26

We can just use four threads and get 100% utilization

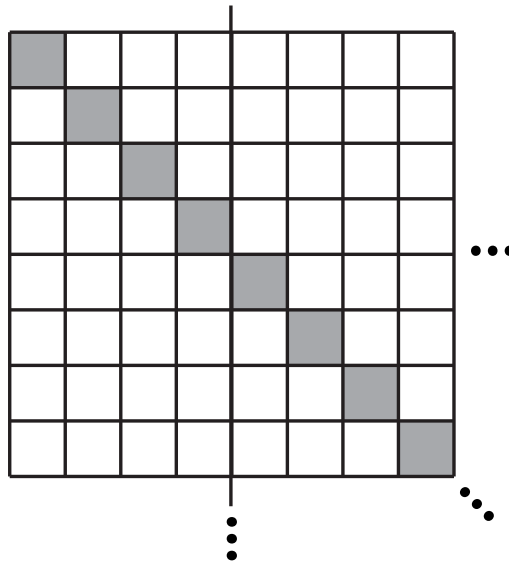
4

What's an equation?



- A. Consider running the code on the image below. **Note the full image is 16×16 pixels... the entire image is not shown in the figure but you can assume the pattern repeats as indicated, with only the diagonal being colored.**

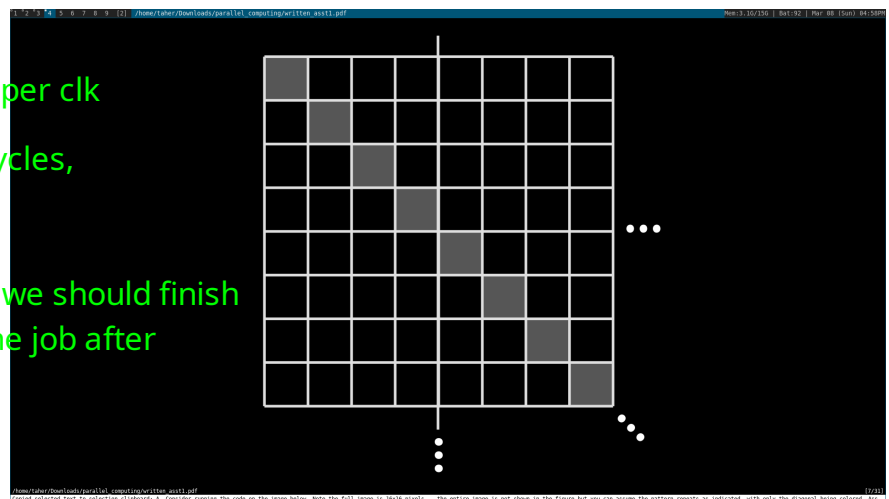
Assume that the only arithmetic cycles we want you to count are the arithmetic instructions labeled in the code. (All conditionals and load/stores are “free”.) What is the overall SIMD efficiency (50%, 100%, etc.) achieved when running the code on a 1 GHz single-core CPU with a **SIMD-width (and corresponding ISPC gang size) of 4**. In other words, the core can perform one 4-wide SIMD operation per clock. (Hint: start by computing the number of cycles needed to perform one row worth of the computation.)



freq. = 1 GHz, single-core, 4-wide SIMD ops per clk

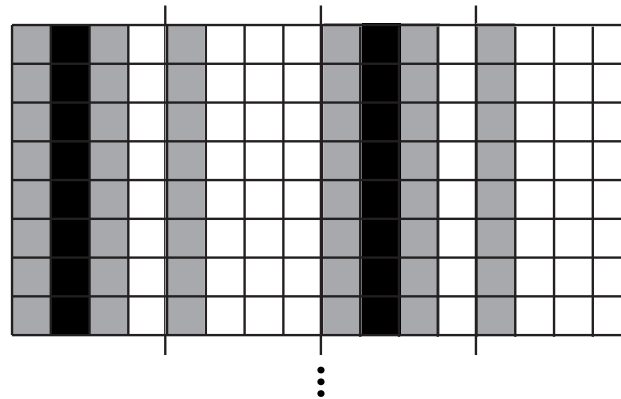
each iteration in the inner loop costs us 10 cycles,
(loads, stores and conditionals are free)

since we do 4 bytes (4 pixels) every iteration. we should finish
a row after 4 iterations, so we should finish the job after
 $16 \times 4 = 64$ iterations (640 clock cycles)



but, for every four iterations we do in parallel we've one iteration that has a pixel wants to go to another
fuction, so we'll do masking here, this will cost extra 10 cycles every 4 iterations (every row), so this costs
160 extra clock cycles for the whole image. So, finally our efficiency is $640 / (160 + 640) = 80\%$

- B. What is the SIMD efficiency obtained when running the same code, using the same processor, but on the image below? Again, please keep in mind the image is 16×16 in size (e.g, the pattern of columns below just extends downward another 8 rows).



we'll do masking twice for the first 4 pixels, once for the second 4 pixels, the same with the right half

so, $(20 + 10) * 2 = 60$ clock cycles every row, then we get
 $60 * 16 = 960$ clock cycle for the whole image

$$\text{efficiency} = 640 / (960 + 640) = 40 \%$$

```

const int IMAGE_SIZE = 16;
void myfunction(uniform float* input, uniform float* output) {

    for (uniform int row=0; row<IMAGE_SIZE; row++) {
        for (uniform int col=0; col<IMAGE_SIZE; col+=programCount) {

            int idx = row*IMAGE_SIZE + col + programIndex;
            float val = input[idx];    // load four bytes
            float result;

            if (!isWhite(val)) {
                if (isGray(val)) {
                    result = foo1(val);    // 10 cycles of arithmetic
                } else {
                    result = foo2(val);    // 10 cycles of arithmetic
                }
            } else {
                result = foo3(val);    // 10 cycles of arithmetic
            }
            output[idx] = result;    // store four bytes
        }
    }
}

```

for each row

```

for row = 0
col = 0 -> idx = programIndex
col = 1 -> idx = programIndex + 1
col = 2 -> idx = programIndex + 2
col = 3 -> idx = programIndex + 3

```

for row = 1

```

col = 0 -> idx = IMAGE_SIZE + programIndex
col = 1 -> idx = IMAGE_SIZE + programIndex + 1
col = 2 -> idx = IMAGE_SIZE + programIndex + 2
col = 3 -> idx = IMAGE_SIZE + programIndex + 3

```

for row = 2

```

col = 0 -> idx = 2 IMAGE_SIZE + programIndex
col = 1 -> idx = 2 IMAGE_SIZE + programIndex + 1
col = 2 -> idx = 2 IMAGE_SIZE + programIndex + 2
col = 3 -> idx = 2 IMAGE_SIZE + programIndex + 3

```

C. Now imagine that you are given the following function:

`transpose(uniform float* input, uniform float* output)`

This function transposes the 16×16 input image and stores the result in a 16×16 output. (Recall that a transpose is $\text{output}[i][j] = \text{input}[j][i]$.) Write down pseudocode for how you would use calls to `myfunction` and `transpose` to produce a computation that eliminates all execution divergence when running `myfunction` on the image from part B. Yes, you can allocate temp buffers as needed.

Fill in pseudocode in `myNewFunction` below. Your solution should compute the same output image output as `myfunction(input, output)`.

```

void myNewFunction(float* input, float* output) {

```

```

    float val = transpose(input[idx]); // val is divergence free

```

```

    result = myfunction(val);
    output = transpose(result)

```

```

}

```

Claude solution :

```

void myNewFunction(float* input, float* output) {

```

```

    float* tmp1 = allocate(16 * 16);

```

```

    float* tmp2 = allocate(16 * 16);

```

```

    // Step 1: Transpose input so vertical stripes → horizontal stripes

```

```

    transpose(input, tmp1);

```

```

    // Step 2: Run myfunction with zero divergence (each row is now uniform color)

```

```

    myfunction(tmp1, tmp2);

```

```

    // Step 3: Transpose result back to restore correct output orientation

```

```

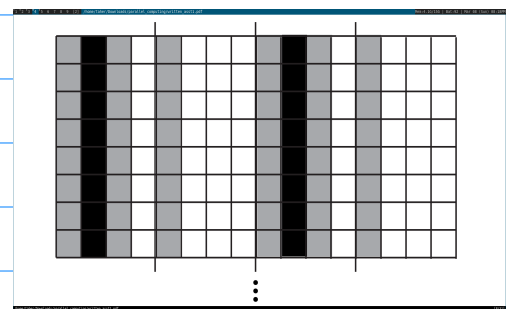
    transpose(tmp2, output);

```

```

}

```



D. Imagine that the code is run the same processor, but now with a memory system with a memory bandwidth of 8 bytes per clock. Please assume that the transpose operation is bandwidth bound and the cost of the operation is equal to the time needed to transfer required data to and from memory. You should also assume that myfunction is compute bound and that the required time is a function of the number of cycles of arithmetic performed.

When running on the input image from part B Is the solution you proposed in part C faster or slower than the single call to myfunction used in part B? How much faster is it (in clocks)?

Please keep in mind that the processor:

- Can execute one four-wide SIMD operation per clock
- Is attached to a memory bus that can transfer 8 bytes of data per clock
- The input/output images are 16×16 elements in size.
- Each image element (a float) is 4 bytes

Hint: compute the amount of time needed to transpose the data, and then compute the amount of time (in clocks) needed to perform myfunction on this modified input.

B.W. = 8 bytes per clock, four-wide SIMD op./clk, the I/O are 16×16 elements, each element is 4 bytes

case | B

we've to bring 4 bytes each clk, and B.W. gives us 8.

So, if each SIMD works on 4×8 bytes, we need 2 clock cycles per row, but things doesn't go as well

so, we've to mask 2 times at the first 4 pixels then we've to mask one time at the second 4 pixels

so, we need $(3 \text{ clk cycles} + 2 \text{ clk cycles}) \times 2 = 10 \text{ clk cycles per row} \rightarrow 160 \text{ clk cycle per row} = 1600 \text{ clk-cycle tot.}$

case | C

we've to move data first, our B.W. is 8 bytes per clk(2 pixels) this'll take $16 \times 16 / 2 = 128$ cycles for read same for write so, we've 256 cycles

from part it'll be optimum 640 cycle (diveragance free)

then we'll transpose it back, this will need 256 cycles

total = 1152 cycles

So, $C/B = 1152/1600 = 0.72 \rightarrow C$ is 1.3888888889 faster than B

PRACTICE PROBLEM 1:

A. Consider a multi-core processor that has two cores. Each core runs at 1 GHz (1 billion operations per clock). Each core is single-threaded (meaning it only maintains state for a single execution context) and can complete one single-precision floating point arithmetic operation per clock. What is the peak arithmetic throughput of the processor in terms of floating point operations per second?

2 cores, $f = 1\text{GHz}$ each peak throughput = $2 * 1\text{GHz} = 2\text{ G op./clk}$ or 2 GFLOPS

B. Now imagine the cores from part A are upgraded so that they perform 16-wide SIMD instructions. Assuming these cores still complete one of these SIMD instructions per clock, what is the peak arithmetic throughput of the processor (in terms of floating point operations per second)?

The new peak throughput = $16 * \text{old throughput} = 32\text{ G op./clk}$ or 32 GFLOPS

C. Finally, imagine that each core from part B was a multi-threaded core that maintain execution contexts for up to four hardware threads each. What is the peak arithmetic throughput of the processor in terms of floating operations per second?

It'll be the same, Again exe. contexts(registers) helps to keep us operate at the max. throughput

D. Imagine that each core from part C was further modified to support superscalar execution where the core can complete one scalar floating point operation and one 16-wide SIMD instruction per clock from the same thread (if those instructions are independent). What is the peak arithmetic throughput of the processor (in terms of floating point operations per second)?

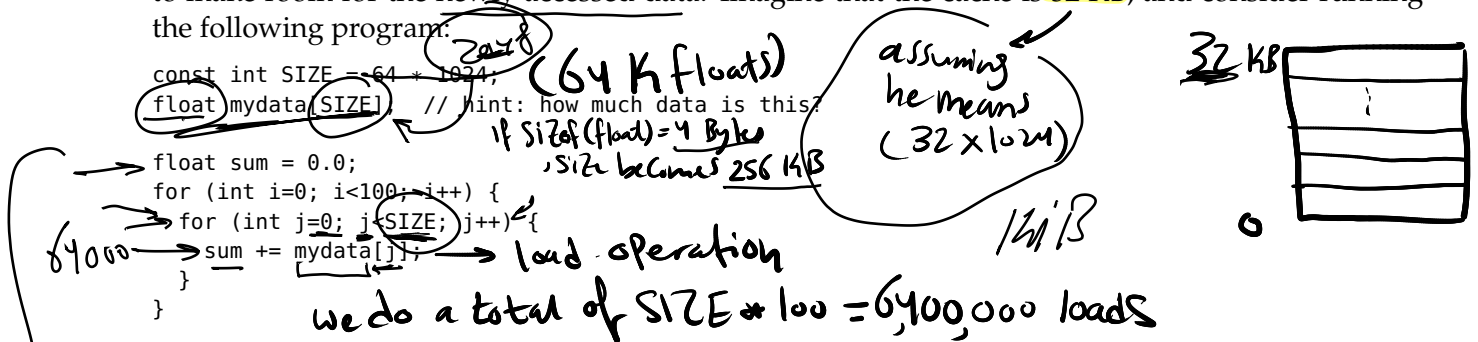
each core was supporting 16-wide SIMD instruction/clk already, now we add another exe. unit that can complete one scalar floating point operation, maybe this speed-up +1 to each core
So, the peak throughput = $32+2 = 34\text{ GFLOPS}$

Caching Basics

PRACTICE PROBLEM 2:

- A. Assume we are running a program on a processor with a **data cache**. All data loaded from memory is first loaded into the processor's data cache, and then transferred from the **cache** to the processor's **registers**. (This is true of most systems.) **When new data is brought into the cache, the cache has a policy of evicting the least recently used data** (the data that has been accessed the longest time ago) to make room for the newly accessed data. Imagine that the cache is **32 KB**, and consider running the following program:

```
const int SIZE = 64 * 1024;
float mydata[SIZE] // hint: how much data is this?
float sum = 0.0;
for (int i=0; i<100; i++) {
  for (int j=0; j<SIZE; j++) {
    sum += mydata[j];
  }
}
```



A **cache miss** occurs when a processor accesses data from memory that is not present in the cache. When the program starts running, **each of data accessed during the $i=0$ iteration of the outer loop is a cache miss**, since that is the first time the data was accessed in the program. Now consider the entire program's execution. Please describe what fraction of the accesses to the mydata array will be cache misses. (For those that are familiar with the details of cache operation, please assume that the cache is **fully-associative**, and has a **cache line size of one float**. If these terms are unfamiliar to you, you can safely ignore them... or Google it!)

cold miss

First 64000 loads miss.

first

Smallest chunk of data to get / leave the cache

First 32 are cold misses **Second 32 are cold misses** **Second 64000 loads**

MISS RATE = 100%

first 32 K see mydata $[j + \frac{\text{SIZE}}{2}]$ so they also miss

- B. Now consider the case where $\text{SIZE} = 2048$. Does your answer to part A change with this new array size. Why or why not?

Yes. Now the current size is 2 K floats (8 KB)

Now on the first "i" iteration, we miss all because of cold misses, but then we don't miss at all because all data reside in the cache. Miss rate is thus $\frac{1}{100} = 1\%$

first iteration

loop unrolling

C. Now assume that you are running the following code. Would you rather have a processor with a 32 KB data cache, or would you rather add multi-threading to the processor. Why or why not?

```
float sum = 0.0;
```

```
for (int i=0; i<1000000; i++) {  
    sum += 2.0 * myarray[i] + 1.0;  
}
```

→ A million iterations

→ Stan?

for for Let's see. Adding a 32 KB cache (32000 Bytes) will get us nowhere (100% miss rate).

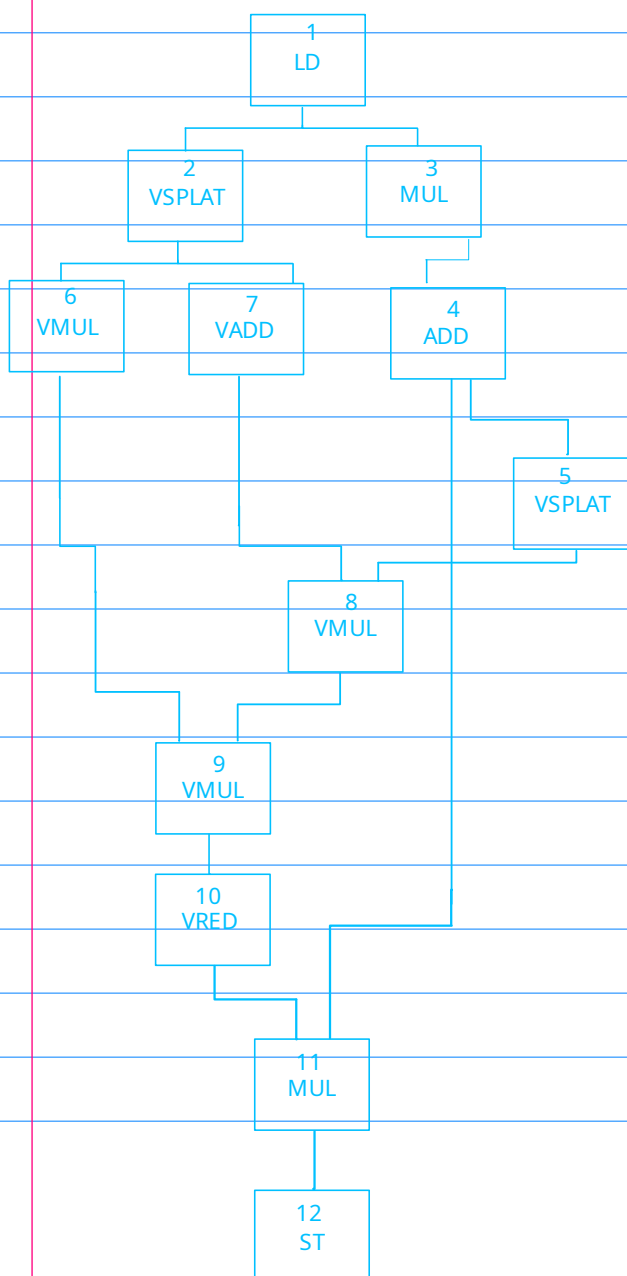
Multi-threading can hide memory latency. Because the threads interleave in execution, and we'll probably only need a number of threads close (or maybe 10) than the miss penalty.

PRACTICE PROBLEM 3:

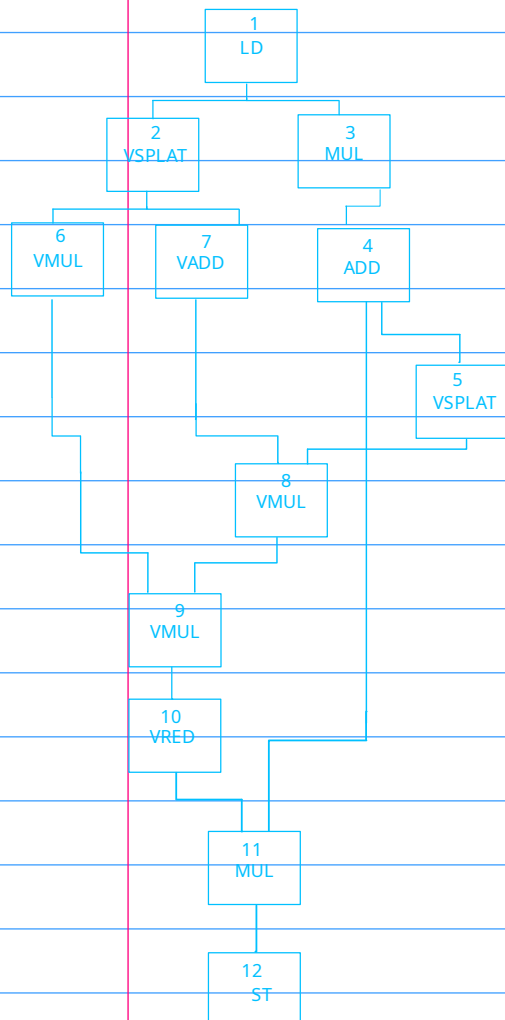
Consider the following sequence of 12 instructions. There is a load operation, followed by 10 math operations, followed by a store. Note that some of the instructions are scalar instructions, and others are vector instructions operating on vector registers (Vx registers). The vector operations have “V” at the beginning of their instruction names.

```
1. LD      R1    <- [R0]
2. VSPLAT  V0    <- R1          // copy R1 into all elements of V0
3. MUL     R2    <- R1, R1
4. ADD     R2    <- R2, 16      // R2 + 16
5. VSPLAT  V1    <- R2          // copy R2 into all elements of V1
6. VMUL    V2    <- V0, V0
7. VADD    V3    <- V0, V0
8. VMUL    V3    <- V1, V3
9. VMUL    V3    <- V2, V3
10. VRED   R1    <- V3          // reduction: sum all elements of V3 into R1
11. MUL    R1    <- R1, R2
12. ST     [R4]  <- R1
```

A. Please draw the dependency graph for the instruction sequence.



B. Imagine the instruction sequence is executed on a single-core, single-threaded processor. The processor supports superscalar execution in that it can fetch/decode up to two instructions per clock, but it has one scalar execution unit and one vector execution unit. Therefore, it can run instructions IN ANY ORDER THAT RESPECTS THE PROGRAM DEPENDENCY GRAPH, but it can only run two independent instructions per clock if and only if one instruction is a scalar instruction and the other instruction is a vector instruction. Assuming that all instructions take 1 cycle to complete, how many cycles does it take to complete this instruction sequence?



1 will take a clk-cycle

2,3 will take a clk-cycle

4,6 will take a clk-cycle

7 will take a clk-cycle

5 will take a clk-cycle

8 will take a clk-cycle

9 will take a clk-cycle

10 will take a clk cycle

11 will take a clk-cycle

12 will take a clk-cycle

total = 10 clk-cycles

C. Now assume the sequence of instructions on the previous page is run in a loop. For example:

```
float in[VERY_BIG]; // let VERY_BIG = 100,000,000
float out[VERY_BIG];
// parallelize iterations of this loop using threads
#pragma omp parallel for
for (int i=0; i<VERY_BIG; i++) {
    // assume myfunc() is the 10 non-LD/ST instrs in the seq above
    out[i] = myfunc(in[i]);
}
```

The loop is run on the same single core, single-threaded processor as before, but now the latency of a LD instruction is 20 clocks. (That is, if the LD instruction begins on clock c , an instruction depending on the LD can begin on clock $c + 20$. (There is one cycle to execute the LD instruction on the processor's scalar unit, followed by 19 cycles of waiting before the dependent instruction can begin.) All other instructions still have a latency of 1 cycle.

In this setup, what is the utilization of the core's vector execution unit? (What fraction of cycles is the vector unit executing instructions? Hint: What are all the reasons the vector unit might not be utilized? Note: it's fine to give your answer as a fraction.)

for each iteration we need to bring a new value(in[i]) -> process it -> store it(out[i])

the non-LD/ST instrs in the seq above took 8 clk-cycles to execute. So, if one LD costs 20 cycles now the total becomes 8+20 or 8+20*2 (IDK, really if we'll count for ST or not)

in the old setup the utilization of the vector exe. unit was 7/10

where $10 = 8(\text{for arithmetic}) + 2(\text{for moving data})$

Now, it would be $7/(9+20) \rightarrow 7/29 = 24.14\%$

D. Now imagine the core is multi-threaded, and can choose to simultaneously execute two instructions from the same thread, or from different threads, provided they meet the one scalar and one vector instruction per clock constraint. In this design, can you achieve full utilization of the vector unit? If so, how many total threads do you need (please explain why) If not, explain why.

Yes, if we've another thread that fills the vector exe. unit while we doing LD/ST. I mean we need to hide 2 cycles of latency then we need two threads. If the latency is 20 as C we need 20 threads

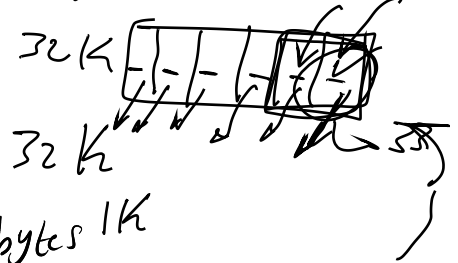
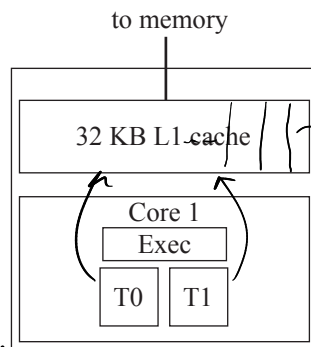
A Task Queue on a Multi-Core, Multi-Threaded CPU

PRACTICE PROBLEM 4:

The figure below shows a single-core CPU with an **32 KB L1 cache** and execution contexts for up to **two threads of control**. The core executes threads assigned to contexts **T0-T1** in an **interleaved fashion** by switching the active thread only **on a memory stall**; **Memory bandwidth is infinitely high in this system, but memory latency on a cache miss is 200 clocks.**

FAQ about the cache: To keep things simple, **assume a cache hit takes only a one cycle**. Assume cache lines are **4 bytes (a single floating point value)**, and the cache implements a **least-recently used (LRU)** replacement policy—meaning that when a **cache line needs to be evicted**, the line that was last accessed the furthest in the past is evicted. It may be helpful to think about **how** this cache behaves when a program reads **33 KB** contiguous bytes of memory over and over. Hint: **confirm to yourself that in this situation every load will be a cache miss.** → *make sense I guess*

In this problem assume the CPU performance no prefetching.



Assume cache is started cold

→ first 32 K reads from cache will miss due to cold conflict

→ That last 1 KB will cause capacity misses (replace the LRU lines)

→ When we read from the start we miss because of the above eviction.

You are implementing a **task queue** for a system with this CPU. **The task queue is responsible for executing independent tasks that are created as a part of a bulk launch** (much like how an ISPC task launch creates many independent tasks). You implement your **task system** using a pool of worker threads, all of which are spawned at **program launch**. When tasks are added to the task queue, the worker threads grab the next task in the queue by **atomically** incrementing a shared counter **next_task_id**. Pseudocode for the execution of a worker thread is shown below.

```
mutex queue_lock;
int next_task_id; // set to zero at time of bulk task launch
int total_tasks; // set to total number of tasks at time of bulk task launch
float* task_args[MAX_NUM_TASKS]; // initialized elsewhere
```

```
while (1) {
```

```
    int my_task_id;
```

```
    LOCK(queue_lock);
```

```
    my_task_id = next_task_id++;
```

```
    UNLOCK(queue_lock);
```

```
    if (my_task_id < total_tasks)
```

```
        TASK_A(my_task_id, task_args[my_task_id]);
```

```
    else
```

```
        break;
```

```
}
```

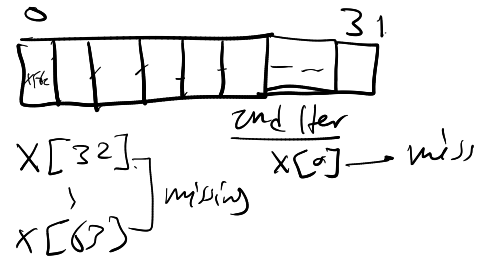
32 KB cache

A. Consider one possible implementation of TASK_A from the code on the previous page:

```
function TASK_A(int task_id, float* X) {
    for (int i=0; i<1000; i++) {
        for (int j=0; j<1024*64; j++) {
            load X[j] // assume this is a cold miss when i=0
            // ... 50 non-memory instructions using X
        }
    }
}
```

250
→ 50
250

we're always gonna miss
misses [X[0]
X[31]



The inner loop of TASK_A scans over 64K elements = 256 KB of elements of array X, performing 50 arithmetic instructions after each load. This process is repeated over the same data 1000 times. Assume there are no other significant memory instructions in the program and that each task works on a completely different input array X (there is no sharing of data across tasks). Remember the cache is 32 KB.

In order to process a bulk launch of TASK_A, you create two worker threads, WT0 and WT1, and assign them to CPU execution contexts T0 and T1. Do you expect the program to execute substantially faster using the two-thread worker pool than if only one worker thread was used? If so, please calculate how much faster. (Your answer need not be exact, a back-of-the envelop calculation is fine.) If not, explain why.

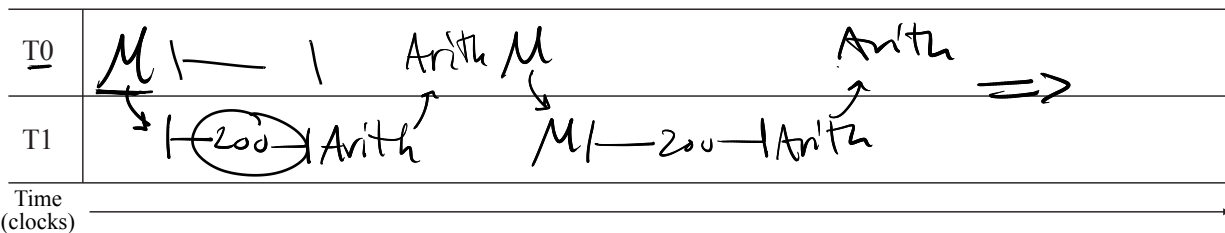
(Careful: please consider the program's execution behavior on average over the entire program's execution ("steady state" behavior). Past students have been tricked by only thinking about the behavior of the first loop iteration of the first task.) It may be helpful to draw when threads are running and stalled waiting for a load on the diagram below.

So a 32KB Cache. A 4 byte (one float) cache line.
A 200 clock miss penalty. 2 threads

With one thread, we execute 50 compute instructions and stall 200 cycles. In two threads, some of that waiting is spent doing 50 compute instructions on another thread

```
function TASK_A(int task_id, float* X) {
    for (int i=0; i<1000; i++) {
        for (int j=0; j<1024*64; j++) {
            load X[j] // assume this is a cold miss when i=0
            // ... 50 non-memory instructions using X
        }
    }
}
```

Case of 1 Thread 256 KB
T0 Miss Arith Miss Arith
T1 Miss Arith Miss Arith



OX



- B. Consider the same setup as the previous problem. How many hardware threads would the CPU core need in order for the machine to maintain peak throughput (100% utilization) on this workload?

Probably a total of 4 threads

$\rightarrow$ for 100% utilization, a stat of 200 cycles and 50 cycles for arithmatics we'd need $\frac{200}{50} = 4$ threads, + 1 = 5 threads total

how much $\swarrow$ arith instr

- C. Now consider the case where the program is modified to contain 100,000 instructions in the inner-most loop. Do you expect your two-thread worker pool to execute the program *substantially faster* than a one thread pool? If so, please calculate how much faster (your answer need not be exact, a back-of-the envelop calculation is fine). If not, explain why.

No. This is a pure memory bottleneck.

Increasing the number of instructions in the loop doesn't affect anything.

```
function TASK_A(int task_id, float* X) {
    for (int i=0; i<1000; i++) {
        for (int j=0; j<1024*64; j++) {
            load X[j] // assume this is a cold miss when i=0
            // ... non-memory instructions using X
        }
    }
}
```

100,000

D. Now consider the case where the cache size is changed to 1 MB and you are running the original program from Part A (50 math instructions in the inner loop). When running the program from part A on this new machine, do you expect your two-thread worker pool to execute the program *substantially faster* than a one thread pool? If so, please calculate how much faster (your answer need not be exact, a back-of-the envelop calculation is fine). If not, explain why.

So Yeah this should remove the bottleneck. First iteration is Cold Misses but starting from second, there is no misses at all. Both Worker one's array and worker two's array fit in the cache $[256 \text{ KB}] \times 2 = 512 \text{ KB}$

Miss rate = 0.1% $\rightarrow$ 99.9% faster than the initial version $\sim \frac{1}{1000}$

$= 512 \text{ KB}$
fits inside 1 MB

T0	
T1	
Time (clocks)	

E. Now consider the case where the L1 cache size is changed to 384 KB. Assuming you cannot change the implementation of TASK_A from Part A, would you choose to use a worker thread pool of one or two threads? Why does this improve performance and how much higher throughput does your solution achieve?

Now 384 KB is bigger than 256 KB but less than 512 KB.
Using a single thread, we can fit the whole array in cache.
However using two threads leads to more misses.

How about throughput?

ops/byte Before $\rightarrow \frac{50 \text{ ops}}{250} = \frac{1}{5} \text{ ops}$

Now $\rightarrow \frac{50 \text{ ops}}{51} \approx 1 \text{ ops}$

5x Throughput

PRACTICE PROBLEM 5:

You write a bit of ISPC code that modifies a grayscale image of size 32×height pixels based on the contents of a black and white “mask” image of the same size. The code brightens input image pixels by a factor of 1000 if the corresponding pixel of the mask image is white (the mask has value 1.0) and by a factor of 10 otherwise.

The code partitions the image processing work into 128 ISPC tasks, which you can assume balance perfectly onto all available CPU processors.

```
void brighten_image(uniform int height, uniform float image[], uniform float mask_image[])
{
    uniform int NUM_TASKS = 128;
    uniform int rows_per_task = height / NUM_TASKS;
    launch[NUM_TASKS] brighten_chunk(rows_per_task, image, mask_image);
}

void brighten_chunk(uniform int rows_per_task, uniform float image[], uniform float mask_image[])
{
    // 'programCount' is the ISPC gang size.
    // 'programIndex' is a per-instance identifier between 0 and programCount-1.
    // 'taskIndex' is a per-task identifier between 0 and NUM_TASKS-1
    // compute starting image row for this task
    uniform int start_row = rows_per_task * taskIndex;
    // process all pixels in a chunk of rows
    for (uniform int j=start_row; j<start_row+rows_per_task; j++) {
        for (uniform int i=0; i<32; i+=programCount) {
            int idx = j*32 + i + programIndex;
            int iters = (mask_image[idx] == 1.f) ? 1000 : 10;
            float tmp = 0.f;
            for (int k=0; k<iters; k++)
                tmp += image[idx]; // these are the ops we want you to count
            image[idx] = tmp;
        }
    }
}
```

divide the work,
launch the tasks

task = SIMD instruction

keeping operations in 32-bit single-precision (float) avoids the overhead of converting to 64-bit (double). Without the f (i.e., just 1.0), the compiler would treat it as a 64-bit double-precision floating-point number (double) by default.

You go to the store to buy a new CPU that runs this computation as fast as possible. On the shelf you see the following three CPUs on sale for the same price:

- (A) 1 GHz single core CPU capable of performing one 32-wide SIMD floating point addition per clock
- (B) 1 GHz 12-core CPU capable of performing one 2-wide SIMD floating point addition per clock
- (C) 4 GHz single core CPU capable of performing one floating point addition per clock (no parallelism)

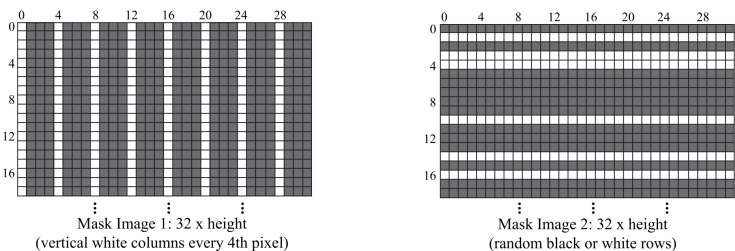


Figure 1: Image masks used to govern image manipulation by brighten_image

A. If your only use of the CPU will be to run the above code as fast as possible, and assuming the code will execute using mask image 1 above, rank all three machines in order of performance. Please explain how you determined your ranking by comparing execution times on the various processors. When considering execution time, you may assume that (1) the only operations you need to account for are the floating-point additions in the innermost 'k' loop. (2) The ISPC gang size will be set to the SIMD width of the CPU. (3) There are no stalls during execution due to data access. (Hint: it may be easiest to consider the execution time of each row of the image.)

for image 1

for A, gang-size = 32, 1 GHz, single core, 1FLOP

Masking is needed once, and masking would take 1000 clk-cycle assuming each addition op. takes 1 cycle

total per row = 1000+10 = 1010 clk-cycles

for B, gang-size = 2, 1GHz, 12 core, 1FLOP

Masking is needed 16 times = 1010 then, the total per row = 8*1000 + 8*10 = 8,080 clk-cycles
then no masking for 16 times = 10

for C, gang-size = 1, 4GHz, single core

total per row = 8 * 1000 + 24*10 = 8,240 clk-cycles

Now, let's calc. the time it would take for each machine

for A: 1010/1*1GHz = 1010 ns then B is faster than A which is faster than C

for B: 8,080/12*1GHz = 673.3333 ns

for C: 8,240/1*4GHz = 2060 ns

unrolling for into ifs can help us get the idea more better

Be a Parallel Processor Architect

PRACTICE PROBLEM 6:

You are hired to start the parallel processor design team at Lagunita Processors, Inc. Your boss tells you that you are responsible for designing the company's first shared address space multi-core processor, which will be constructed by cramming multiple copies of the company's best selling uniprocessor core on a single chip. Your boss expects the project to yield at least a 5x speedup on the performance of the program given below. You are not allowed to change the program, and assume that:

- Each Lagunita core can complete one floating point operation per clock (1 flop/clock)
- Cores are clocked at 1 GHz, and each have a 1 MB cache using LRU replacement.
- All Lagunita processors (both single and multi-core) are attached to a 100 GB/s memory bus
- Memory latency is perfectly hidden (Lagunita processors have excellent pre-fetchers)

float A[N]; // let N = 100 million elements
float total = 0;

// ASSUME TIMER STARTS HERE

for (int i=0; i<N; i++)
total += A[i];

for (int i=0; i<9; i++) {

// made up syntax for brevity: 'parallel_for'

// Assume iterations of this loop are perfectly partitioned
// using blocked assignment among X pthreads each running on
// one of the processor's X cores.

parallel_for (int j=0; j<N; j++) {
A[j] = A[j] / total;

}

// ASSUME TIMER STOPS HERE

Amdahl's Law

Performance after Parallelization
is limited by how much
can be Parallelized

I'm guessing this can't be parallelized much

A[j]

total
A[j] ← Store

One approach for the second loop is to use 10 cores each doing an instance of the inner loop. Each of these cores access memory at the same time. We have a memory BW of 100 GB/s

If each core is at 1 GHz

Each inner loop iteration does

2 mem op → 2x4 = 8 Bytes of mem traffic

10 instances

for (100 million times) {
LD R1 ← A[j]
F.O.P R2 ← R1 / total
SD [R2] ← A[j]
}

8 byte x 9 x 10⁶ 72 GB/s
 Roughly 3 Instructions → 333 x 10⁶ transaction/s
 So 333 x 10⁶ $\frac{\text{trans}}{\text{sec}}$ x 8 $\frac{\text{Bytes}}{\text{trans}}$ = 2.67 GB/s per core
 every one of them does 8 Bytes of mem traffic

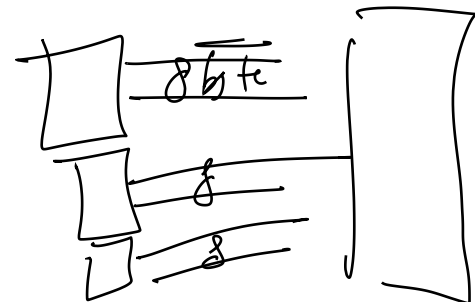
A. How do you respond to your boss' request? Do you believe you can meet the performance goal? If yes, how many cores should be included in the new multi-core processor? If no, explain why.

Well, let's see. Amdahl's law states

$$S = \frac{1}{(1-P) + \frac{P}{N}}$$

Parallelizable
no core

$$5x = \frac{1}{(1-90\%) + \frac{90\%}{N}} \rightarrow N = 9$$



$$\frac{1}{5} = 0.1 + \frac{0.9}{N}$$

$$\frac{0.9}{0.1} = N, N = 9$$

• So we can. And the mem BW used is less than

26 GB/s, so we're fine

B. You tell your boss that if you were allowed to make a few changes to the code, you could deliver a much better speedup with your parallel processor design. How would you change the code to improve its performance by improving speedup? (A simple description of the new code is fine). If your answer was NO in part one, how many processors are required to achieve 5x speedup now? If your answer was YES, approximately what speedup do you expect from your previously proposed machine on the new code? (Note: we are NOT looking for answers that optimize the program by rolling multiple divisions into one.)

Maybe we can improve the only portion that cannot be parallelized,

Think about it as splitting the N iterations to 10 smaller $\frac{N}{10}$ and make each thread work on one portion.

(my idea → Claude's CORRECT implementation)

```
float partial[10] = {0}; // partial sums, one per thread
float total = 0;
par [0] ++
par [1] ++

// This CAN be parallel - each thread works on its own chunk
parallel_for(int i=0; i<N; i++) {
  for (int j = i*(N/10); j < (i+1)*(N/10); j++)
    partial[i] += A[j];
}
```

```
10 x 90000 x 100,000,000 LG
// This cannot be parallel - but only 10 additions!
for (int i=0; i<10; i++)
  total += partial[i];
```

How much performance do we get using 9 cores?

Using Amdahl's law again (almost 100% of the program is parallelizable)

$$X = \frac{1}{(1-P) + \frac{P}{N}} = 9 \rightarrow 9x \text{ Performance}$$

Page 26

$$9 - \frac{9}{5} = 80\% \text{ Improvement}$$

C. Assume that the following year, Lagunita Processors, Inc. decides to produce a 32-core version of your parallel CPU design. In addition to adding cores, your boss gives you the opportunity to further improve the processor through one of the following three options.

1. You may double each processor's cache to 2 MB.

2. You may increase memory bandwidth by 50% → 150 GB/s

3. You may add a 4-wide SIMD unit to the core so that each core can perform 4 floating point operations per clock. FLOPS

If each of these options has the same cost, given the code you produced in part B (and what you learned from assignment 1), which option do you recommend to your boss? Why?

Code → what is that?

$N=9$

9x

1. He didn't mention anything about cache misses which, even what I only know for sure that we have a hypothetical zero-latency memory. A larger cache will probably do nothing as the dominant part of the program (second loop) uses N mem access per loop which is (far) bigger than the 2 MiB.
2. I think bandwidth isn't a problem at all we are already using about 25% of the original 100 GB/s
3. This is definitely it! I mean the whole program is operating on an array. Now we can group 4 FP loads, 4 FP div, or FP stores together. This requires $8 \times 4 = 32$ bytes of memory access per transaction. That's 4 times the BW needed from before. Now, for the 9 cores we're using, we get $4 \times 2.67 \text{ GB/s} \times 9 = 97 \text{ GB/s}$ (still doable)